Northeastern University Spring 2026

Multimodal Semantic Retrieval Engine

Late Fusion Two-Tower Model on Amazon Berkeley Objects

Adithya Ramesh, Hrishikesh Pradhan, and Radhika Khurana

The Problem We Are Trying To Solve

User finds a product they like online or in the real world. They would like to find similar products using an image and a free form text description or structured text attributes like color, material, brand, style etc.

Why It's Hard

- Image and text are different modalities. By default, they will give embeddings that live in different spaces.
- Fine-grained visual features (texture, material) are not well-captured by general-purpose models
- We have to search in a large catalog of products.

Dataset: Amazon Berkeley Objects (ABO)

147K	398K	573	9
Product listings	Catalog images	Product types	Structured attribute fields

STRUCTURED ATTRIBUTE FIELDS

- 🏷️ **Brand** — Rivet, AmazonBasics
- 🎨 **Color** — White, Walnut, Navy
- 🪵 **Material** — Oak, Leather, Steel
- 🛋️ **Product Type** — Sofa, Bar Stool, Lamp
- ✨ **Style / Finish** — Mid-Century, Rustic
- 📐 **+ 4 more** — size, weight, pattern...

TEXT DESCRIPTION CONSTRUCTION

Item Name + Product Type + Color: X + Material: Y + Style: Z →

```
"Rivet Sawyer Side Table. Side Table. Color: Walnut. Material: Wood. Style: Mid-Century Modern."
```

WHY ABO?

📷 **Production-realistic images**
Clean, white-background catalog shots — the exact distribution a deployed retrieval system sees, not scraped web images.

🏷️ **Attribute richness by design**
Structured fields per listing make attribute-constrained retrieval genuinely evaluable — not retrofitted from captions.

⚖️ **Right scale for research**
147K items is large enough to stress-test retrieval across 573 product types, yet tractable on a single GPU.

EXPERIMENTAL SPLIT

Gallery (Train) — 117,702 · 80%

Queries (Test) — 29,425 · 20%

~205	224^2	16
avg items / product type	input resolution	listing files sampled

EVALUATION CHALLENGES

! Multilingual attribute labels
"White" is stored as Beyaz, Blanc, or Blanco depending on locale. Symbolic matching breaks; neural models must generalize.

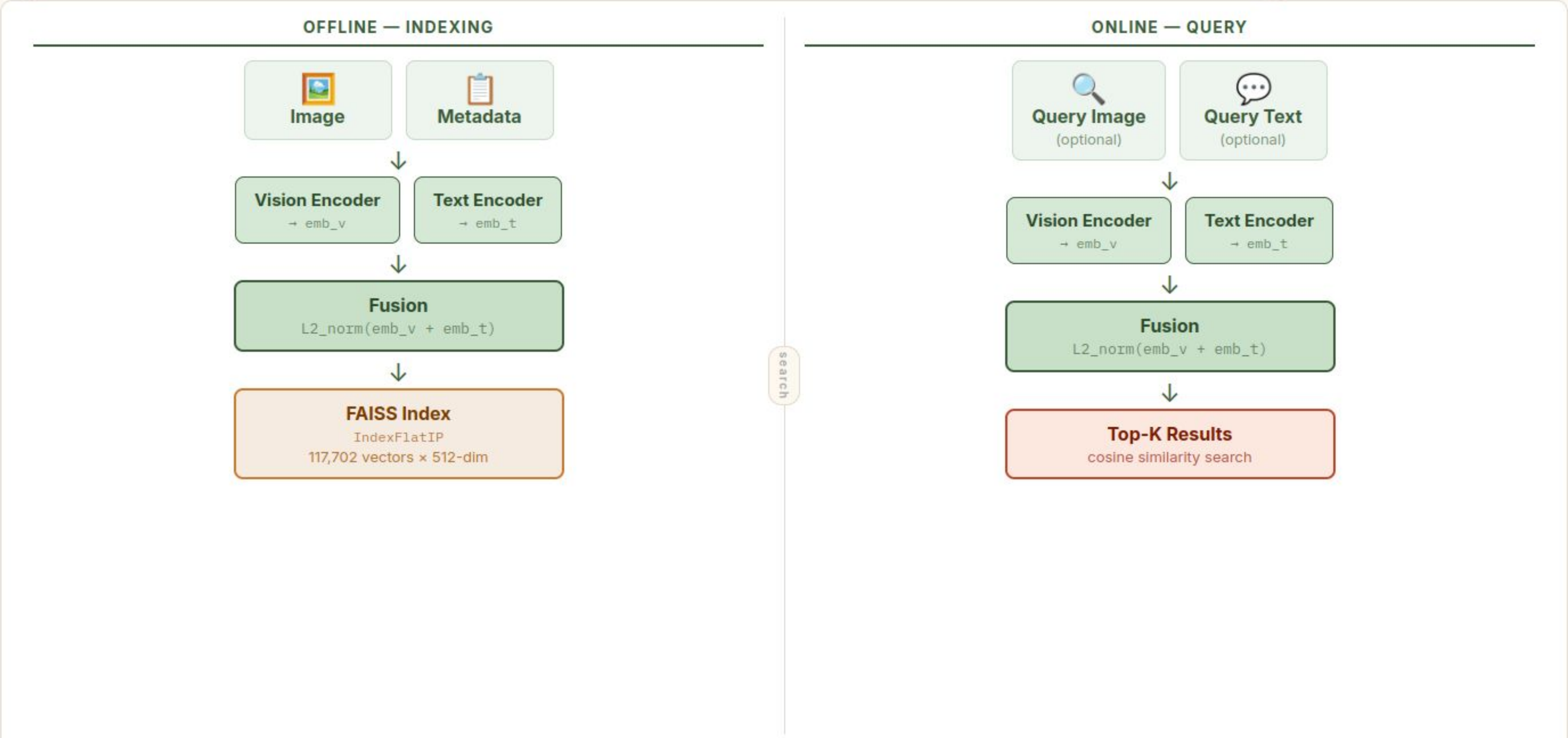
! Sparse attribute coverage
Fewer than 500 items carry the color "White" across 30K listings. Sparse queries produce structurally weak text embeddings.

! Fuzzy product type boundaries
"Accent chair" and "lounge chair" overlap semantically. P@K treats them as wrong answers — introducing an irreducible noise floor.

! Recall ceiling is arithmetic
With ~500 sofas in 148K items, R@10 is capped at 0.02 by definition. Low recall numbers reflect catalog density, not model quality.

Ground truth: items sharing the same `product_type` are treated as relevant — a category-level proxy, not instance-level annotation.

System Architecture — Multimodal Semantic Retrieval Engine



EVALUATION METRICS

Precision@K K ∈ {1, 5, 10}
`P@K = (relevant in top-K) / K`

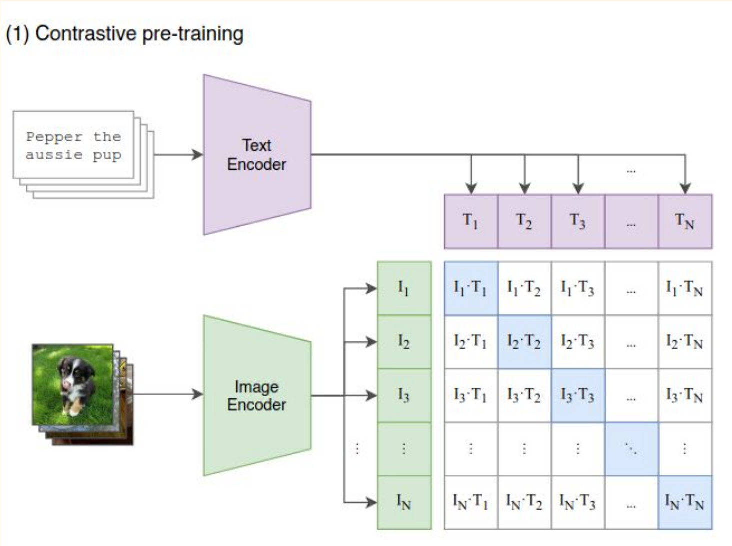
Recall@K K ∈ {1, 5, 10}
`R@K = (relevant in top-K) / (total relevant)`

mAP
`mAP = Σ P@k · rel(k) / Σ rel(k)`

CLIP: Architecture & Baseline

CLIP (Contrastive Language-Image Pretraining) was trained by OpenAI on 400 million image-text pairs scraped from the internet. It learns a shared embedding space for images and text. It tries to ensure that semantically related images and text produce similar embeddings.

Architecture



- Image encoder: ViT-B/32 vision transformer → 512-d image embedding
- Text encoder: Transformer (63M params) → 512-d text embedding
- Both encoders project into the same L2-normalized embedding space
- Trained to minimize contrastive loss:
  - Minimize cosine similarity b/w wrong (image, text) pairs
  - Maximize cosine similarity b/w correct (image, text) pairs

Pre-trained CLIP as a Baseline

- CLIP is known to provide strong out of the box performance on image and text data
- We use pre-trained CLIP to establish a baseline on ABO before any domain-specific adaptation

Metric	Pretrained CLIP
Precision @ 1	0.9138
Precision @ 5	0.8832
Precision @ 10	0.8623
Recall @ 1	0.0032
Recall @ 5	0.0121
Recall @ 10	0.0200
mAP	0.9170

Fine-Tuning Strategy 1: Fine Tune Projection Head on ABO

CLIP's visual and text representations are already strong - a small learned transformation may be enough to adapt them to ABO's specific distribution. So, we freeze the backbone of the encoders and fine tune only the projection head. This is highly parameter efficient and we avoid catastrophic forgetting.

01

Freeze backbone

Freeze the backbone of the image and text encoders

02

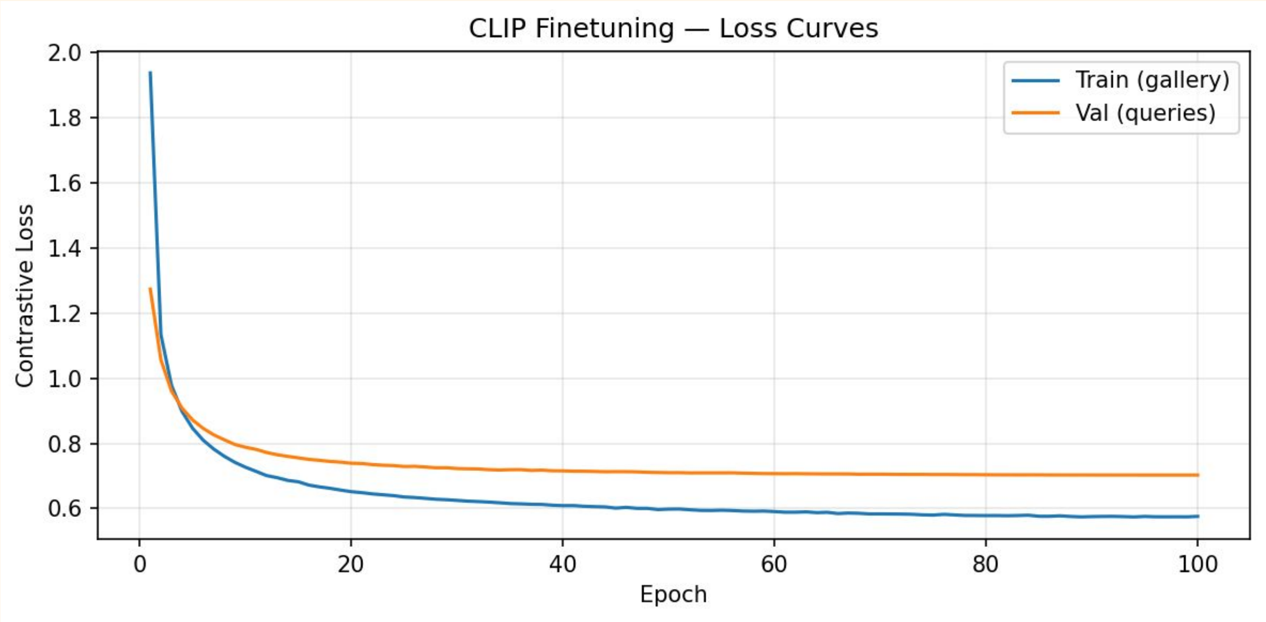
Fine Tune Projection Head

Train only the projection head (linear layer) on top of each encoder. Only 655,360 / 151,277,312 parameters (0.4%) are trained.

03

Train on ABO

Minimize contrastive loss on ABO image-text pairs. Trained for 100 epochs on 148K catalog.



Metric	Proj Head FT CLIP
Precision @ 1	0.9124
Precision @ 5	0.8838
Precision @ 10	0.8640
Recall @ 1	0.0033
Recall @ 5	0.0126
Recall @ 10	0.0209
mAP	0.9174

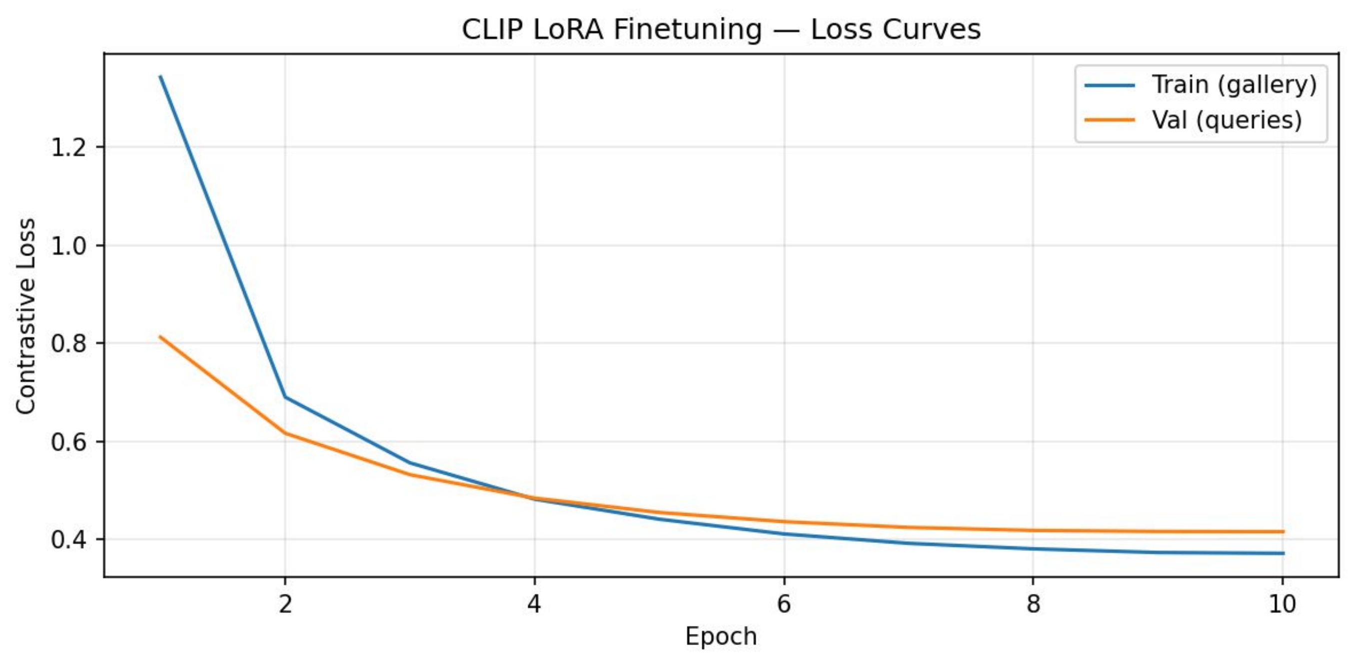
Fine-Tuning Strategy 2: LoRA on CLIP

In Low-Rank Adaptation (LoRA) we freeze the original pre-trained weights and train small, low-rank matrices added to specific layers. This is parameter efficient and we avoid catastrophic forgetting.

How LoRA Works

For each weight matrix W (frozen), add a trainable bypass: W' = W + B A, where, $B \in \mathbb{R}^{d\times r}, A \in \mathbb{R}^{r\times s}, \mathrm{rank}(A) = r \ll d$

- Applied to the query and value projection matrices in each transformer attention block
- Rank r = 16 used here: ~0.6% of CLIP's total parameters are trainable
- At inference, B A is merged back into W - zero added latency



Metric	LoRA FT CLIP
Precision @ 1	0.9108
Precision @ 5	0.8798
Precision @ 10	0.8581
Recall @ 1	0.0032
Recall @ 5	0.0124
Recall @ 10	0.0205
mAP	0.9142

CLIP as a Baseline: Stronger Than Expected

Pretrained CLIP performs well on the ABO dataset right out of the box. Surprisingly, fine-tuning CLIP (both projection head fine-tuning and LoRA) did not meaningfully improve its performance. This strong out-of-the-box performance is likely due to CLIP being trained on 400 million image-text pairs from the internet, a significant portion of which is probably related to e-commerce, making ABO a relatively in-distribution dataset for CLIP.

Metric	Pretrained CLIP	Proj Head FT CLIP	LoRA FT CLIP
Precision @ 1	0.9138	0.9124	0.9108
Precision @ 5	0.8832	0.8838	0.8798
Precision @ 10	0.8623	0.8640	0.8581
Recall @ 1	0.0032	0.0033	0.0032
Recall @ 5	0.0121	0.0126	0.0124
Recall @ 10	0.0200	0.0209	0.0205
mAP	0.9170	0.9174	0.9142

Key Conclusion: Pretrained CLIP demonstrates strong out-of-the-box performance on the ABO dataset, indicating that fine-tuning techniques like projection head fine-tuning and LoRA do not lead to significant metric improvements.

Two-Tower Approach

Why We Still Built a Custom Architecture

CLIP demonstrates surprisingly strong out-of-the-box performance on the ABO dataset, achieving approximately 0.91 P@1, and interestingly, fine-tuning doesn't meaningfully improve this metric. This raises an honest question: if CLIP performs so well, why invest in building a custom architecture at all?

What P@K Actually Measures

P@K evaluates category-level retrieval: did the system return a sofa for a sofa query?

CLIP's 0.91 P@1 is likely the ceiling for this type of matching on ABO given label noise.

But P@K does NOT measure whether the returned sofa is white, oak, or from brand Rivet

What Our System Was Built For

DINOv3 Image Tower: Fine-grained visual features for texture and material discrimination

Per-Attribute Encoding: Independent encoding prevents attribute dilution via learned attention pooling

Learned Fusion α: Model dynamically balances image vs text contribution

Hybrid Search + Re-ranking: Exact symbolic matching ensures attribute fidelity

Key Takeaway: Comparable P@K numbers don't signify equivalent systems. They often mean both systems solve the easy part (category retrieval). The true challenge, and the focus of our custom architecture, lies in achieving precise attribute fidelity.

Architecture Decision: DINOv3 for the Image Tower

Why DINOv3?

DINOv3 is self-supervised, trained on **1.7 billion images** to learn visual similarity without any text supervision. This means its representations are shaped entirely by visual structure which is not forced to align with coarse language labels.

The result: it captures fine-grained features like **wood grain, fabric texture, and surface finish** that CLIP's vision encoder, optimized for image-text coarse alignment, systematically missed.

Technical Fit

- ViT-B/16 backbone → 768-d CLS token output
- Linear projection head → 512-d shared space
- Drop-in replacement for CLIP vision encoder - identical output dimension
- Self-supervision means no modality entanglement at pretraining

Key Insight: DINOv3's self-supervised training objective explicitly encourages patch-level feature consistency, making it structurally better suited for texture and material discrimination than contrastive image-text pretraining.

Architecture Decision: Attribute-Aware Text Encoding

The attribute dilution failure in CLIP directly motivated a fundamental change to how text was encoded. Instead of treating the full attribute set as a single string, each attribute is encoded **independently** through CLIP's text encoder - preserving its individual signal.

1

Naive Approach

`"brand: Rivet | material: Oak | color: White"`Single CLIP text pass. Short attributes drowned out by longer fields.

2

Per-Attribute Encoding

Three separate CLIP text passes: `"brand: Rivet"`, `"material: Oak"`, `"color: White"`. Each attribute gets its own 512-d representation.

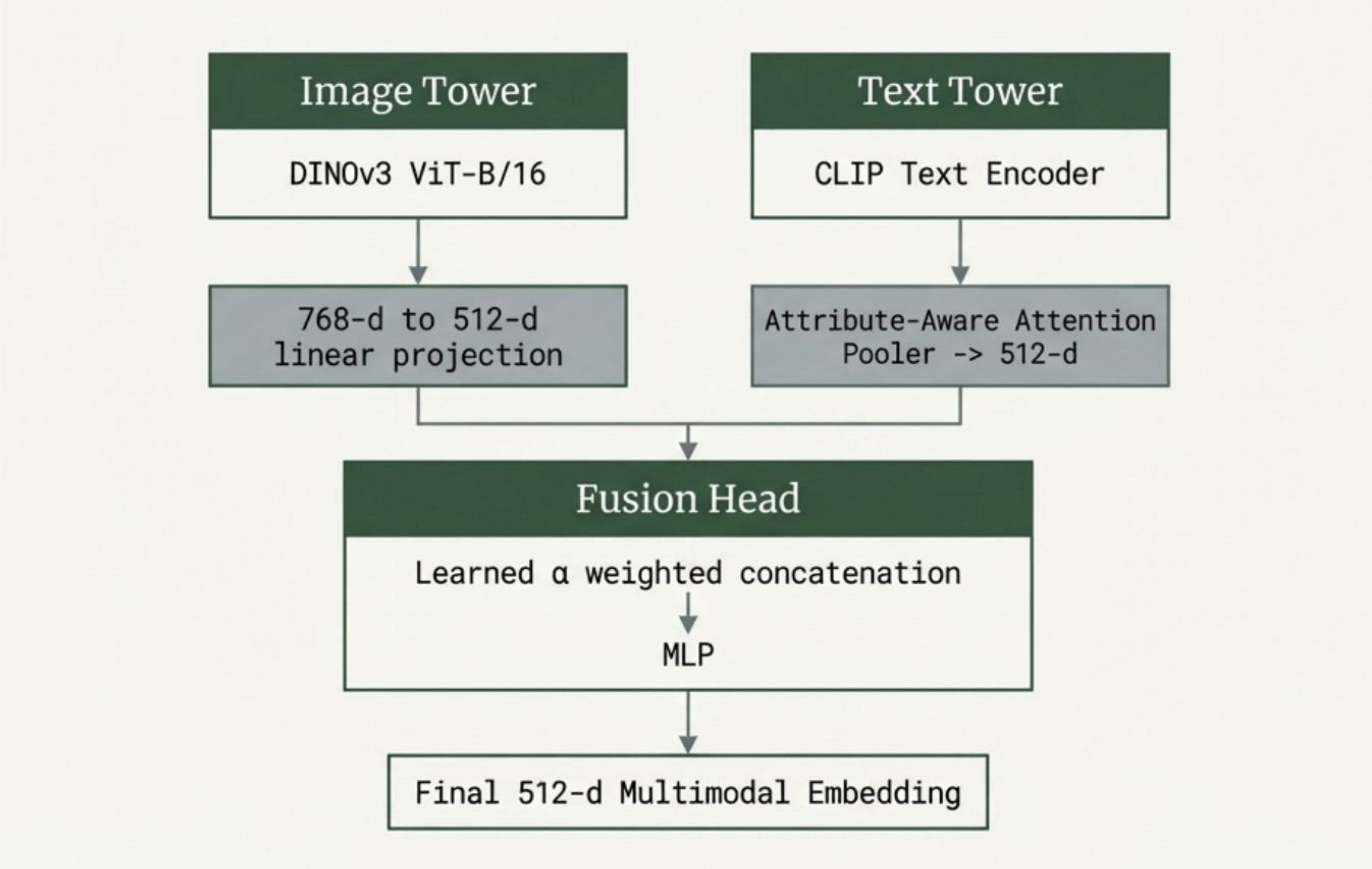
3

Attention Pooler

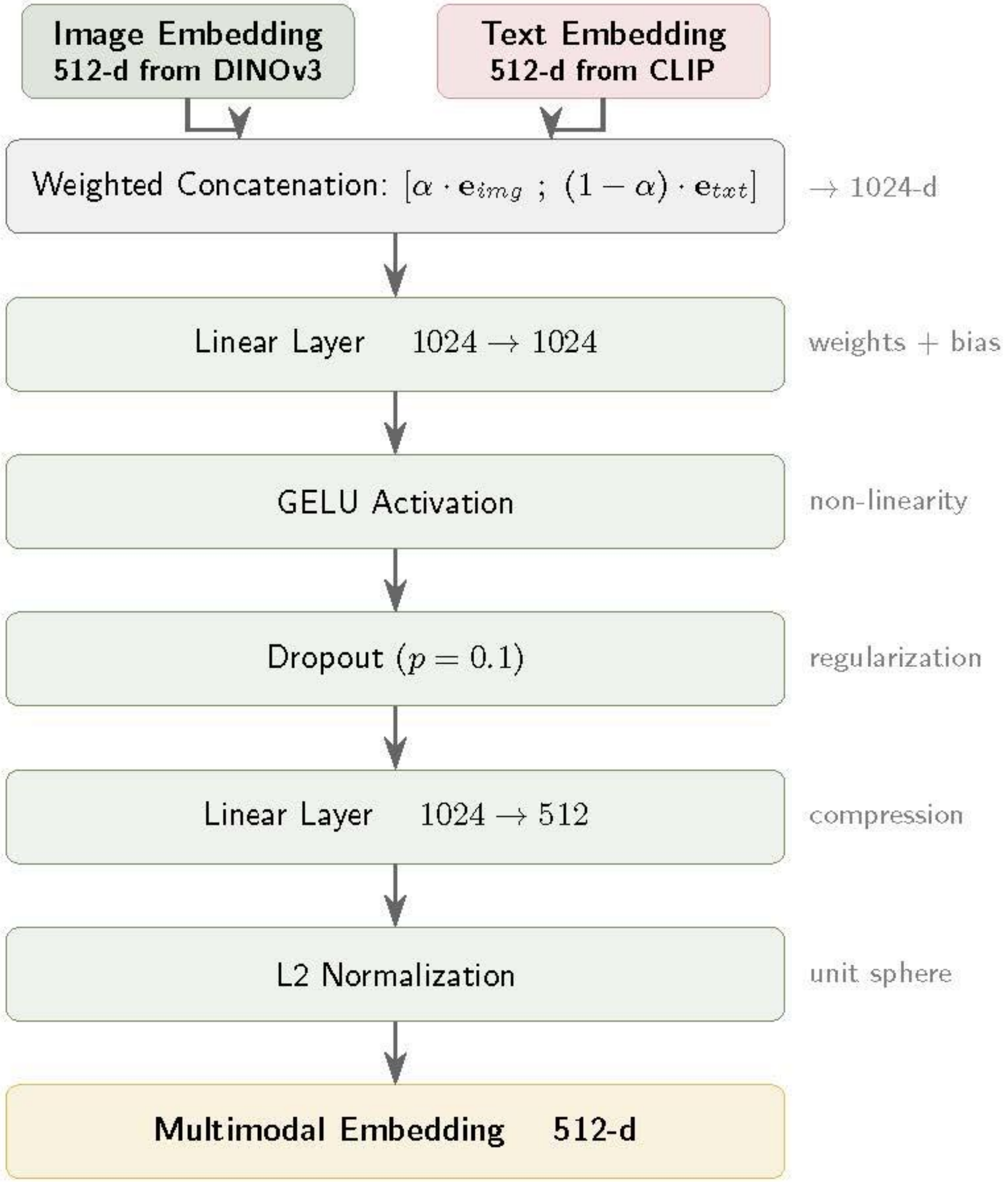
A learned attention pooler aggregates attribute embeddings, discovering which attributes matter most for matching in context. Output: single 512-d text embedding.

This design directly addresses dilution: no attribute can be overshadowed by string length. The attention pooler also provides interpretability, we can inspect learned weights to understand which attributes the model treats as most discriminative.

Full Architecture: Late Fusion Two-Tower Model



Fusion Head MLP



$\alpha = \sigma(learned\_scalar)$, *initialized at 0.0 so* $\sigma(0) = 0.5$

Why Partial Unfreezing - Not Frozen, Not Full Fine-Tune

Backbone freezing strategy was not a hyperparameter search - it was a deliberate decision motivated by inspecting what each layer type actually encodes.

1

Frozen Backbones: Too Rigid

Only linear projections can be learned. The image tower cannot adapt to distinguish oak grain from walnut - DINOv3's pretrained features are close but not calibrated for this catalog's specific texture vocabulary.

2

Full Fine-Tuning: Catastrophic Forgetting

With only 148K training items, full fine-tuning risks overwriting DINOv3's general visual representations, destroying the texture-sensitivity that motivated its selection in the first place.

3

Partial Unfreeze: Targeted Adaptation

Unfreeze only the **last 2 transformer blocks** at a **10× lower learning rate** than the projection head. Early layers (edges, textures) remain frozen; later task-specific layers adapt to fine-grained product differentiation.

Training Strategy: Dual Contrastive Loss

Two contrastive objectives were used simultaneously, each targeting a specific alignment problem — with loss weights tuned based on observed failure modes during training.

Loss 1: InfoNCE on Fused Embeddings

Pushes different products apart in the shared 512-d space. Treats every other item in the batch as a negative. Ensures the multimodal embedding is discriminative across the catalog.

$$L_{InfoNCE} = -\log \frac{\exp(\mathrm{sim}(q_i, k_i)/\tau)}{\sum_j \exp(\mathrm{sim}(q_i, k_j)/\tau)}$$

Loss 2: PairwiseInfoNCE (Cross-Modal Alignment)

Aligns image and text embeddings of the **same product** - directly minimizing the modality gap. Initially weighted at 0.5×, but inspection showed color queries were being systematically ignored. Weight increased to **1.0×** after confirming text-side underrepresentation in the gradient signal.

Diagnostic Motivation: The weight adjustment from 0.5× to 1.0× on the alignment loss was not arbitrary. Embedding space analysis showed cross-modal cosine similarity stagnating at ~0.3 mid-training - the cross-modal signal was being dominated by the within-modal InfoNCE gradient.

Training Strategy: Targeted Data Augmentation

The Distribution Shift Problem

The model performed well on catalog-to-catalog retrieval but degraded significantly on **external query images** (user photos, Google images). Systematic diagnosis revealed the cause: training images are clean, centered, white-background product shots. Real-world queries have variable lighting, partial occlusion, arbitrary crops, and perspective distortion.

Augmentation Strategy

- **RandomCrop**: simulates partial views and off-center framing
- **HorizontalFlip**: orientation invariance for symmetric products
- **ColorJitter**: handles lighting variation without confusing material identity

Applied only to the image tower during training, not at inference. No additional data required.

Diagnosing the Fusion Weight Imbalance

After an initial training run, we audited the learned fusion weight: a single sigmoid-gated scalar α controlling the image/text contribution to the fused embedding. The result revealed a systematic initialization bias.

62%

Image Weight

Learned fusion weight allocated to the image tower after training with α initialized at 0.5

38%

Text Weight

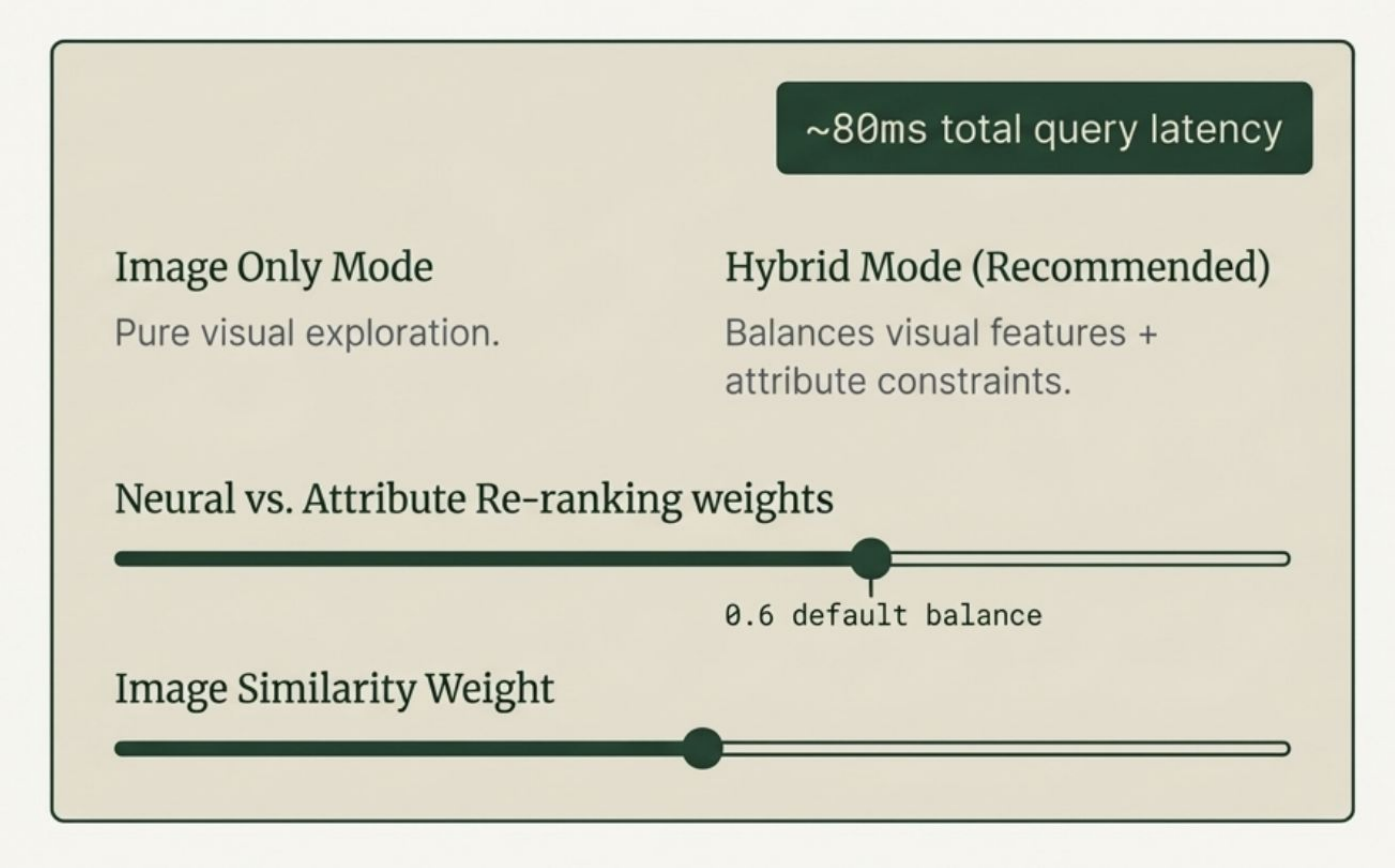
Resulting text tower contribution is insufficient to enforce color and material constraints

0.0

Corrected Init

New initialization value for α. sigmoid(0.0) = 0.50, giving both modalities equal gradient signal from the first step

Root Cause: Initializing α = 0.5 passes through sigmoid to 0.622, not 0.5. This pre-biases the model toward image dominance before a single gradient update. The fix is mechanically simple but diagnostically important: it came from inspecting *why* color attribute queries were consistently underweighted, not from hyperparameter tuning.



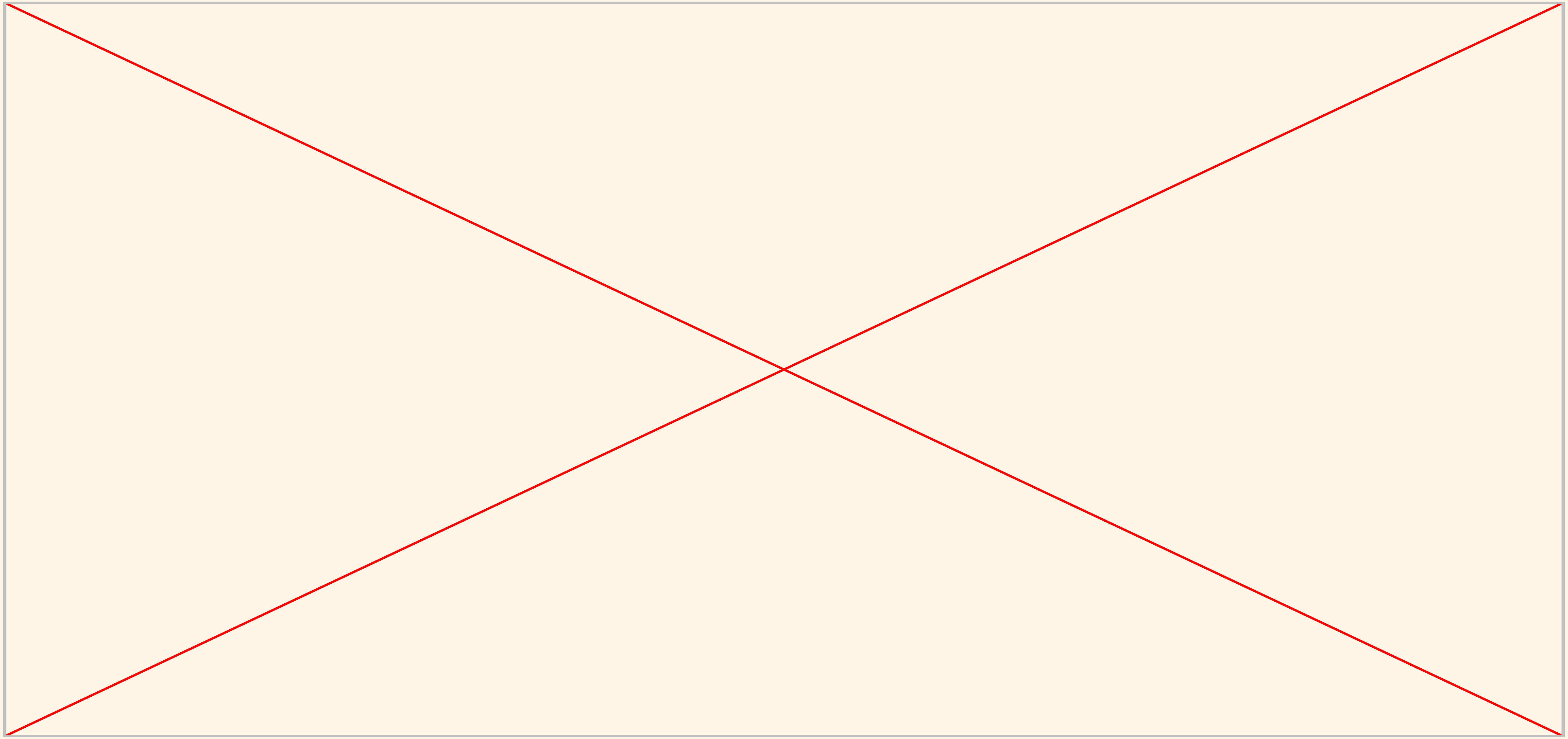
TEXT

IMAGE

FUSED INDEX

IMAGE INDEX

DEMO



Key Results

Evaluation Results: 148K Catalog

91.62%	88.5%	86.3%	91.7%
P@1	P@5	P@10	MAP
Topres ult is the correct product category	At least one correct match in top 5 results	Precision maintained over a 10-result window	Mean Average Precision across all queries

Key Insight: P@K measures category-level retrieval only, not attribute fidelity. Both systems return a sofa for a sofa query. Only the two-tower system can enforce 'white, oak, Rivet brand.' The metrics measure the ceiling we share; the architecture determines what happens beyond it.

On Low Recall@10 (R@10 = 0.02): With ~500 sofas in a 148K catalog, retrieving 10 items gives a theoretical maximum R@10 of 10/500 = 0.02. The model is performing at ceiling for this metric.

Scaling Experiments: 5K → 30K → 148K

Each scaling decision was motivated by specific diagnostic observations, not simply "more data is better." The attention pooler in particular required sufficient attribute diversity to learn meaningful per-attribute weights.

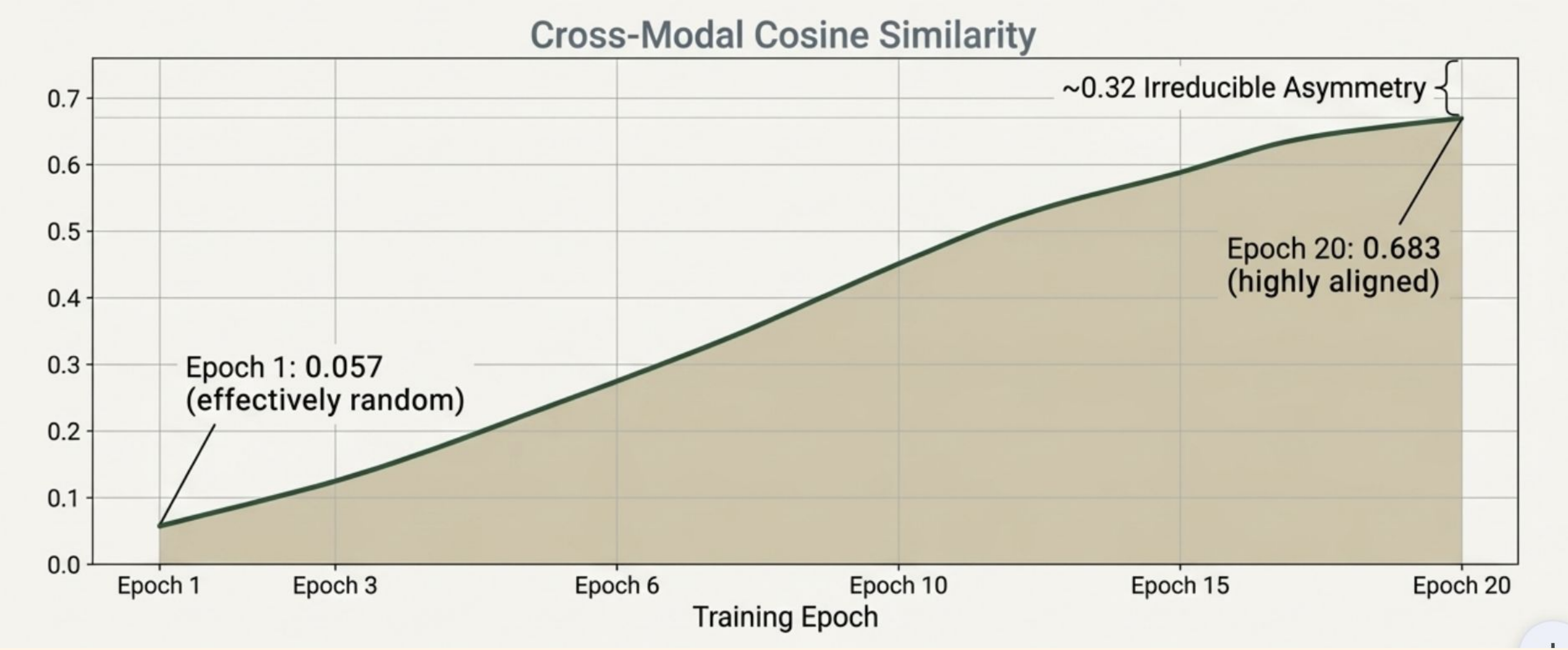
Performance Across Training Scales

Metric	5K Items	30K Items	148K Items
P@1	0.8120	0.8693	0.9162
P@5	0.7596	0.8239	0.8849
P@10	0.7232	0.7904	0.8630
R@1	0.0321	0.0092	0.0028
R@5	0.0848	0.0315	0.0113
R@10	0.1160	0.0496	0.0197
mAP	0.8233	0.8754	0.9177
Val Cosine	0.523	0.640	0.683

At 5K items, a large train/val loss gap indicated the model was underfitting the data distribution, the embedding space was sparse relative to catalog diversity. At 30K, cross-modal alignment improved sharply (val_cos: 0.52 → 0.64), confirming the attention pooler was learning meaningful attribute weights. At 148K, all metrics converged with diminishing marginal gains, suggesting the architecture had reached its practical capacity ceiling.

Modality Gap Analysis: Closing the Embedding Distance

Cross-modal cosine similarity is the most direct measure of whether the two towers have learned to inhabit the same semantic space.



Self-Retrieval Validation: Verifying Embedding Quality

Before attributing demo degradation to the model itself, we performed a targeted sanity check: can each catalog item retrieve **itself** as its own top-1 result? This isolates index quality from query-side distribution shift.

Result: Perfect Self-Retrieval

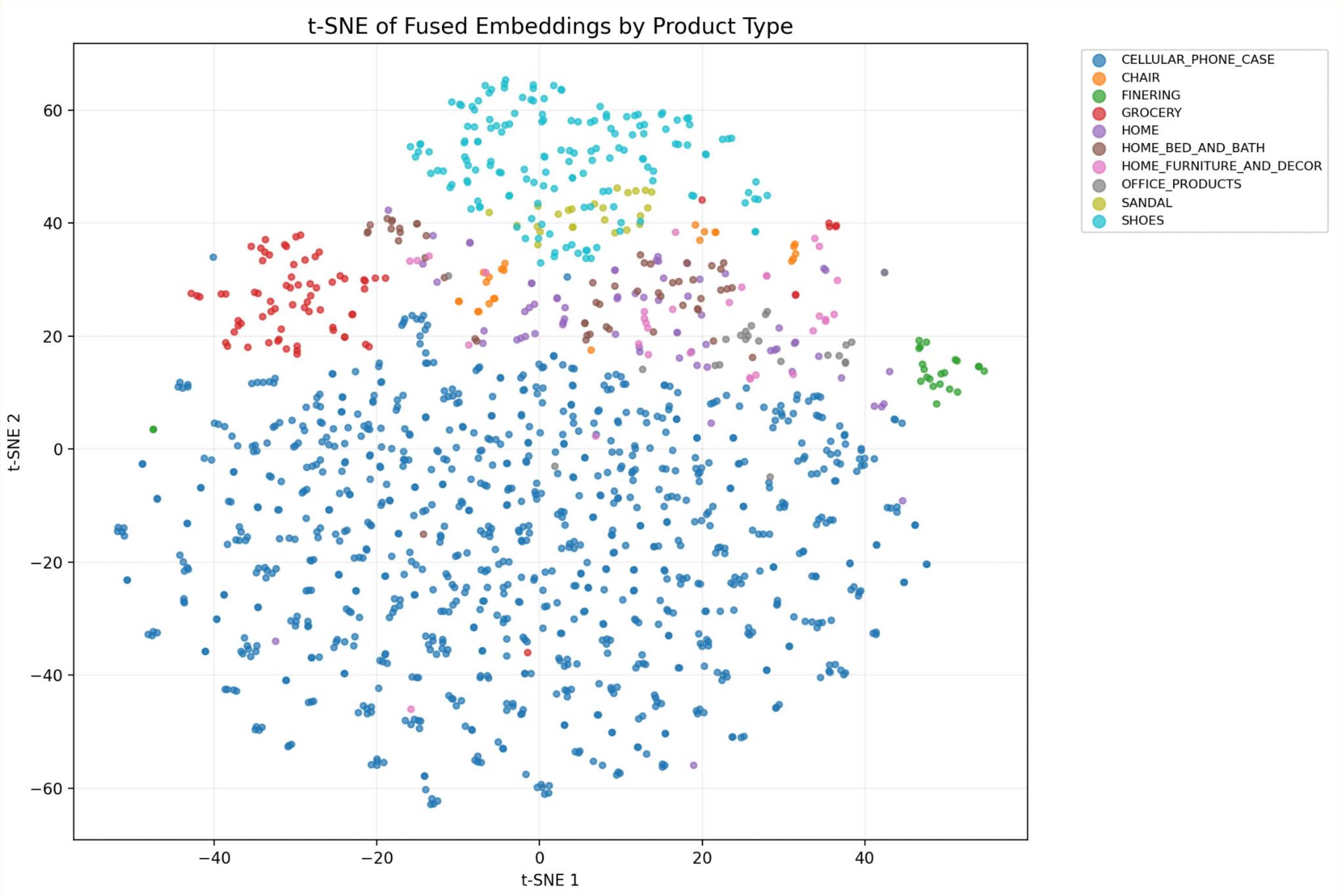
10/10 items tested achieved top-1 self-retrieval with cosine similarity score of **1.0000** across both fused and image-only indexes..

Score Separation

Clean margin between self-similarity (1.000) and nearest-neighbor similarity (0.92–0.98), confirming embeddings are not degenerate and the space is well-calibrated.

Self-retrieval is a necessary but not sufficient condition for good retrieval. It confirms the embedding space has preserved identity, but discriminability between near-neighbors requires the full evaluation suite.

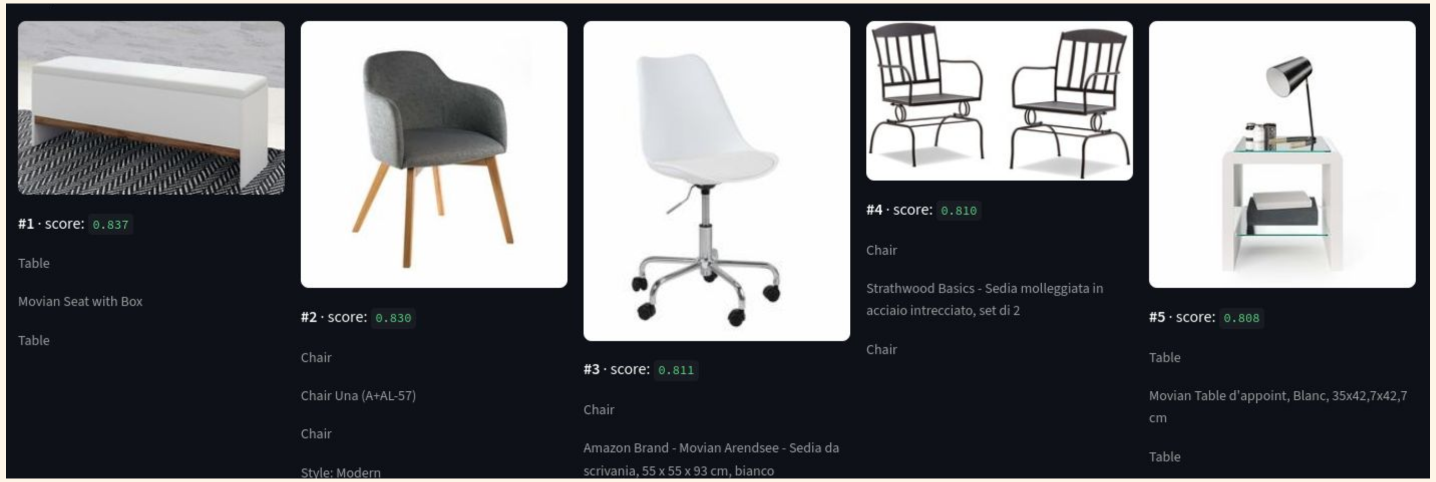
t-distributed Stochastic Neighbor Embedding



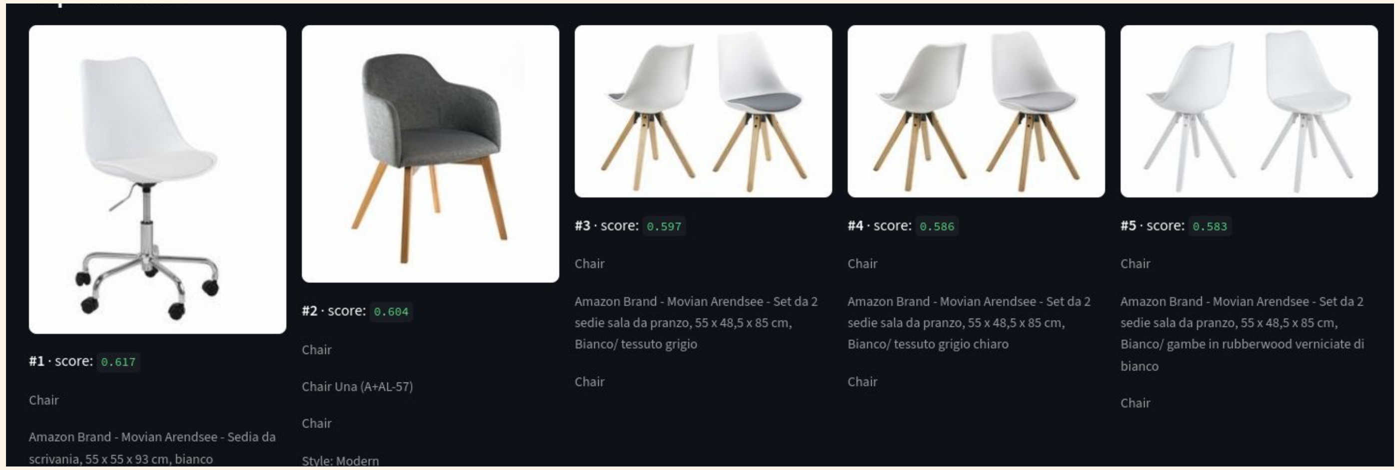
SAMPLE EDGE CASE RESULTS

QUERY:

WHITE METAL CHAIR

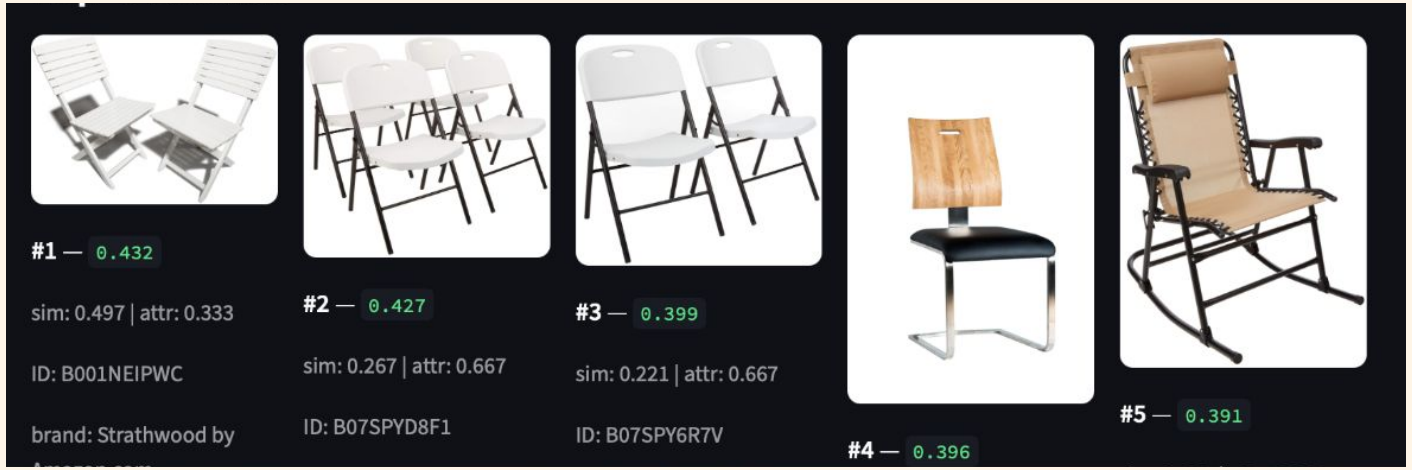


PRE-TRAINED MODEL



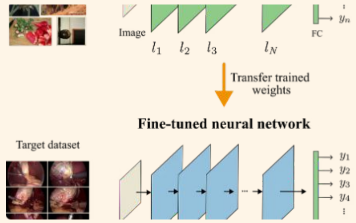
FINE-TUNED MODEL

FUSION-INDEX MODEL



Comparison of Approaches

Three team members pursued different architectural strategies. Each approach reflects a genuine design trade-off optimized for a different primary use case - not a ranking of quality.

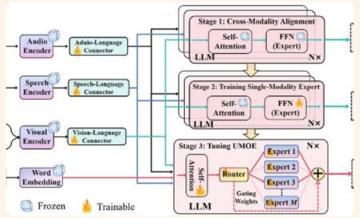


Approach A: CLIP-based Retrieval

Strategy: CLIP-based retrieval, explored with pretrained, projection head fine-tuned, and LoRA fine-tuned variants.

Optimized for: Strong out-of-the-box performance; leveraging powerful general pretraining to match custom architectures on category-level metrics.

Key Finding: Pretrained CLIP achieved P@1 = 0.9138, mAP = 0.9170 out-of-the-box. Fine-tuning did not meaningfully improve performance, likely because CLIP was pretrained on 400M image-text pairs including substantial e-commerce data, making ABO relatively in-distribution.

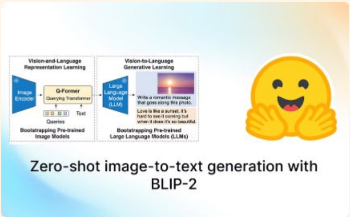


Approach B: Specialized Encoders + Trained Fusion

Strategy: DINOv3 image tower + per-attribute CLIP text encoding + learned fusion.

Optimized for: Explicit user attribute constraints, strongest eval metrics on structured queries.

Trade-off: Requires training; less flexible for image-only queries without augmentation.



Approach C: BLIP Captioning + CLIP

Strategy: Auto-generate text descriptions from images via BLIP, then encode with CLIP.

Optimized for: Image-only queries with no user-provided attributes, better out-of-box visual similarity.

Limitation: Generated captions cannot enforce explicit user constraints.

- sometimes clip alone misses the actual product; blip was seeing a dog when it was a phone case with a dog on it

Key Takeaway: Diagnosis-Driven Architecture

Every architectural decision in this system was a **direct response to an observed, diagnosed failure**, not a result of random experimentation or hyperparameter search.

CLIP → Specialized Encoders

CLIP could not handle fine-grained attribute discrimination → separate DINOv3 image tower + per-attribute text tower

Concatenation → Attention Pooling

Attribute dilution in concatenated strings → independent per-attribute CLIP encoding with learned attention aggregation

Frozen → Partial Unfreeze

Linear projections insufficient; full fine-tune risked forgetting → last 2 blocks unfrozen with 10× differential LR

Biased Init → Corrected α

Sigmoid(0.5) = 0.622, pre-biasing fusion toward images → initialized at $\alpha = 0.0$ for true 50/50 starting balance

Single Index → Hybrid Dual-Index

Demo queries with 1–2 attributes mismatched rich-attribute index → separate image-only index with score-bonus consensus merging

The methodology matters as much as the result. A system built through systematic failure diagnosis is inherently more debuggable, more interpretable, and more amenable to targeted improvement than one built through ablation alone.

Limitations & Future Directions

Current Limitations

- **Dataset Size:** Although our data set is large, for the scope of this problem it is still not nearly enough - we can see this with only a limited set of product types
- **InfoNCE treats all negatives equally:** Same-category different-product items are treated identically to cross-category negatives, weakening the gradient signal for intra-category discrimination.

Prioritized Future Work

01

Hard Negative Mining

Sample negatives from within the same product category to sharpen intra-category discrimination.

02

Cross-Attention Re-Ranker

Second-stage fine-grained matching module that attends jointly over image patches and attribute tokens.